PROGRAMACIÓN ORIENTADA A OBJETOS

Práctica Profesionalizante

Unidad IV: Programación Orientada a Objetos

Contenido Temático

a. Conceptos Básicos.
b. Principios Fundamentales de la POO.

¿Qué es la POO?

Es importante aclarar la diferencia que existe entre programación orientada a objetos y un lenguaje orientado a objetos.

La programación orientada a objetos es un modelo de programación, con su teoría y su metodología. Un lenguaje orientado a objetos es un lenguaje de programación que permite el diseño de aplicaciones orientadas a objetos. Dicho esto, lo normal es que toda persona que vaya a desarrollar aplicaciones orientadas a objetos aprenda primero la "filosofía" (o adquiera la forma de pensar) y después el lenguaje, porque "filosofía" sólo hay una y lenguajes muchos.

Primero es conveniente indicar lo que **NO** es la POO:

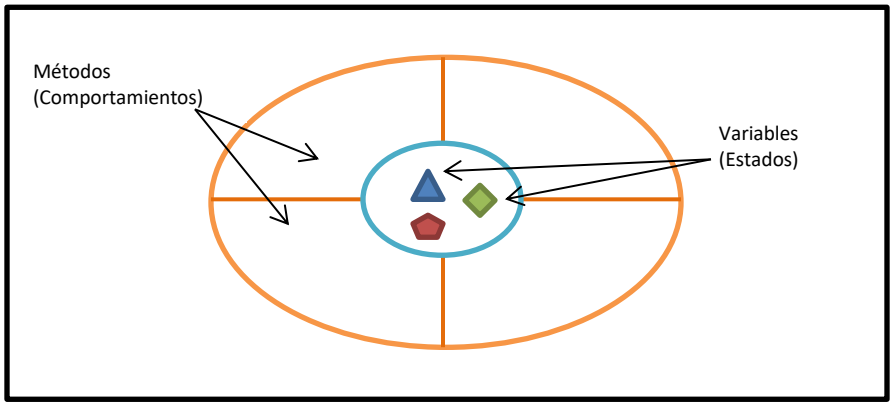
- **NO** es un sistema de comunicación con los programas basados en iconos, ventanas, etc.
- **NO** es un lenguaje. De hecho las técnicas de POO pueden utilizarse en cualquier lenguaje conocido y los que están por venir, aunque estos últimos, al menos en los próximos años, incluirán facilidades para el manejo de objetos. Desde luego, que en los lenguajes que prevén el uso de objetos la implementación de las técnicas de POO resulta mucho más fácil y provechosa que los otros. Pero del mismo modo a lo comentado en el punto anterior, se pueden utilizar estos lenguajes sin que los programas resultantes tengan nada que ver con la POO.

Es así que la **Programación Orientada a Objetos** es un intento para dominar la complejidad que, de forma innata, posee el software. La *programación orientada a objetos* es un nuevo método de descomposición es la *descomposición en objetos*; se focaliza no en lo que hay que hacer en el problema, sino en cuál es el escenario real del mismo, y se intenta simular ese escenario en el programa a desarrollar.

En pocas palabras podemos definir a la **POO** como un paradigma de programación que facilita la creación de software de calidad por sus factores que potencian el mantenimiento, la extensión y la reutilización del software generado bajo este paradigma.

Objetos

Un objeto es una entidad lógica, la representación en un programa de un concepto, y contiene toda la información necesaria para abstraerlo: datos que describen sus atributos y operaciones que pueden realizarse sobre los mismos. La siguiente figura muestra una representación visual de un objeto.



Un objeto consta de:

- **Duración**: La duración de un objeto en un programa siempre está limitada en el tiempo. La mayoría de los objetos sólo existen durante una parte de la ejecución del programa.
- **Estado**: Todo objeto posee un estado, definido por sus *atributos*. Con él se definen las propiedades del objeto, y el estado en que se encuentra en un momento determinado de su existencia.
- **Comportamiento**: Todo objeto ha de presentar una interfaz, definida por sus *métodos*, para que el resto de objetos que componen los programas puedan interactuar con él.
- **Identidad**: Propiedad del objeto que lo distingue de los demás.

El equivalente de un *objeto* en el paradigma estructurado sería una *variable*. Así mismo la *instanciación de objetos* equivaldría a la *declaración de variables*, y el *tiempo de vida* de un objeto al *ámbito* de una variable.

Clases

Las clases son abstracciones que representan a un conjunto de objetos con un comportamiento e interfaz común.

En realidad, *un objeto es una instancia de una clase*, por lo que se pueden intercambiar los términos objeto o instancia.

Una clase se compone de dos partes:

- *Atributos* (denominados, por lo general, datos miembros): esto es, los datos que se refieren al estado del objeto.
- *Métodos* (denominados, por lo general, funciones miembros): son funciones que pueden aplicarse a objetos.

Cómo modelar e instanciar una clase

En el curso vamos a usar la siguiente nomenclatura para la definición de las clases:

NOMBRE: Auto
ATRIBUTOS Puerta Color Marca
MÉTODOS Girar Frenar

De allí podemos *instanciar* de la siguiente forma:

NOMBRE: Auto Uno
ATRIBUTOS Puerta: 4 Color: Roja Marca: Fiat Palio
MÉTODOS Girar() Frenar()

NOMBRE: Auto Dos
ATRIBUTOS Puerta: 2 Color: Negra Marca: Fiat Fire
MÉTODOS Girar() Frenar()

Principios fundamentales de la POO

Introducción

Existen una serie de principios fundamentales para comprender cómo se modela la realidad al crear un programa bajo el paradigma de la orientación a objetos.

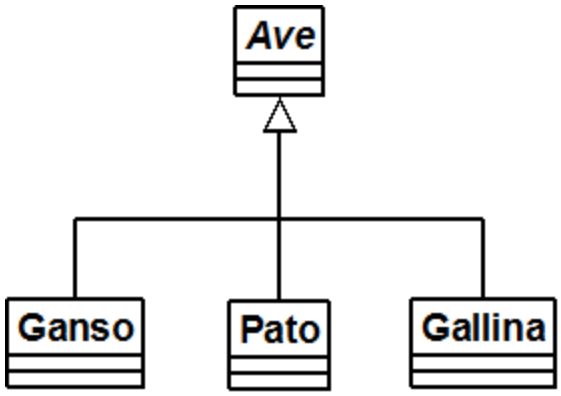
Herencia

Es la relación entre una clase general y otra clase más específica. Es un mecanismo que nos permite crear clases derivadas a partir de la *clase base* o *superclase*, lo que genera el compartir automáticamente métodos y datos entre clases, subclases y objetos. Por ejemplo: Si declaramos una clase párrafo derivada de una clase texto, todos los métodos y variables asociadas con la clase texto, son automáticamente heredados por la subclase párrafo.

La herencia es uno de los mecanismos de la programación orientada a objetos, por medio del cual una clase se deriva de otra, llamada entonces *superclase*, de manera que extiende su funcionalidad.

Las clases se relacionan entre si formando una jerarquía entre ellas. La herencia facilitará el encapsulamiento y el polimorfismo. La herencia es transitiva, esto quiere decir que si la clase A hereda de la clase B y la clase B hereda de la clase C, la clase A hereda los componentes de clase C.

Aquí vemos una representación gráfica de herencia:



Abstracción

La técnica de la *abstracción* consiste en aislar un determinado elemento sobre su contexto o el resto de elementos que lo acompañan y que posiblemente no sean válidos para nuestro propósito. Gracias a la abstracción separamos las características esenciales de un objeto de las no esenciales y tomaremos las características y propiedades a tener en cuenta de un objeto en el mundo real con el fin de utilizarlas en el sistema que tratamos de implementar. Por ejemplo, para el caso de implementar conceptos de *abstracción* en la clase Auto, ¿qué características o propiedades comparten todas las Auto?, esta es la pregunta para poder abstraer correctamente.

Encapsulamiento

El *encapsulamiento* consiste en reunir todos los elementos que pertenecen a una misma entidad y otorgarles un determinado nivel de aislamiento según convenga. El aislamiento protege a los datos asociados a un objeto contra su modificación por actores que no tengan derecho sobre ellos. Gracias a esto el usuario de la clase solo se ha de concentrar en el uso de la misma y no en como realmente funcionan sus métodos.

Este principio otorga beneficios importantes para el desarrollo de software:

- Modularidad: El código fuente para un objeto puede ser escrito y mantenido independientemente del código fuente para otros objetos. Además, un objeto puede ser fácilmente pasado de un lugar a otro del sistema. Por ejemplo, en el caso del Auto de lecturas anteriores, es posible darle tu Auto a alguien más, y continuará trabajando.
- Ocultamiento de información: Un objeto tiene una interfaz pública que otros objetos pueden utilizar para comunicarse con él. El objeto puede mantener información y métodos privados que pueden ser modificados a cualquier tiempo sin afectar los otros objetos que dependen de

él. No es necesario entender cómo usar el mecanismo de los engranajes de una Auto para poder utilizarlo.

¿Cómo requerimos la actuación de un método?

Enviando un *Mensaje* al objeto. Al enviar un mensaje, se ejecuta un método, el cual puede llamar a su vez a otros métodos de su clase o de cualquier otra clase o bien cambiar los valores de los datos de ese objeto. Si se desea alterar los valores de los datos de un objeto deberá mandar un mensaje a ese objeto; lo mismo sucede cuando un objeto tiene que cambiar los valores de los datos de otro objeto.

Como podemos apreciar, un objeto es como una caja negra, a la que se le envía un mensaje y éste responde ejecutando el método apropiado, el cual producirá las acciones deseadas. Un objeto, una vez programado es solo manipulable a través de mensajes.

Polimorfismo

En programación orientada a objetos se denomina *polimorfismo* a la capacidad que tienen los objetos de una clase de responder al mismo mensaje o evento en función de los parámetros utilizados durante su invocación. Un *objeto polimórfico* es una entidad que puede contener valores de diferentes tipos durante la ejecución del programa.

Dicho de otra forma, el polimorfismo consiste en conseguir que un objeto de una clase se comporte como un objeto de cualquiera de sus subclases, dependiendo de la forma de llamar a los métodos de dicha clase o subclases.

En algunos lenguajes, el término polimorfismo es también conocido como *sobrecarga de parámetros* ya que las características de los objetos permiten aceptar distintos parámetros para un mismo método (diferentes implementaciones) generalmente con comportamientos distintos e independientes para cada una de ellas.